*ISFT 189*

Tecnicatura Superior en Análisis de Sistemas

Prácticas Profesionalizantes III

# Manual técnico

*Sistema de Gestión Comercial Prana Electrónica*

Autor Valentín Morello

Año 2026

Edición 2 de octubre de 2026

# Índice

1 Propósito del documento ..... 3
2 Tecnologías y dependencias ..... 3
3 Requisitos de desarrollo ..... 3
4 Estructura de la solución ..... 3
5 Referencias entre proyectos ..... 4
6 Arquitectura por capas ..... 5
7 Convenciones utilizadas ..... 6
8 Configuración de conexión ..... 6
9 Inicializador de base de datos ..... 7
10 Scripts SQL ..... 7
11 Modelo de datos ..... 7
12 Entidades principales ..... 8
13 Servicios de negocio ..... 9
14 Acceso a datos ..... 9
15 Formularios y navegación ..... 9
16 Sesión roles y permisos ..... 9
17 Seguridad con BCrypt ..... 10
18 Transacciones ..... 10
19 Gestión de errores ..... 10
20 Comprobantes HTML ..... 11
21 Pruebas ..... 11
22 Publicación ..... 11
23 Instalador ..... 11
24 Actualización del sistema ..... 12
25 Respaldo y recuperación ..... 12
26 Limitaciones técnicas ..... 12
27 Mantenimiento futuro ..... 12

# 1 Propósito del documento

Este manual describe la organización, persistencia, seguridad, publicación y mantenimiento de Prana Electrónica. Permite localizar las clases que implementan los procesos y reproducir la preparación del sistema sin alterar su información comercial. El código documentado corresponde al estado del proyecto al 2 de octubre de 2026.

La implementación se organiza en cuatro proyectos y una base LocalDB. Las referencias se expresan mediante rutas relativas al repositorio. Las limitaciones y las tareas técnicas se detallan en el anexo; no se consideran funcionalidades completas.

# 2 Tecnologías y dependencias

| Proyecto | Objetivo .NET | Dependencias directas |
|---|---|---|
| Prana.Presentacion | net10.0-windows | Windows Forms; FontAwesome 4.7.0; FontAwesome.Sharp 6.6.0. |
| Prana.negocio | net10.0 | Referencia a Prana.Datos. |
| Prana.Datos | net10.0 | Microsoft.Data.SqlClient 7.0.2; BCrypt.Net-Next 4.2.0; System.Configuration.ConfigurationManager 10.0.10; referencia a Entidades. |
| Prana.Entidades | net10.0 | Modelos de datos sin paquetes externos declarados. |

El proyecto de negocio accede a BCrypt mediante las dependencias transitivas de Datos. La presentación también utiliza las referencias transitivas para el inicializador. Esta distribución permite compilar, aunque la asignación explícita del paquete a la capa que lo usa puede mejorarse.

# 3 Requisitos de desarrollo

Se requiere Windows, un SDK .NET 10 compatible con los proyectos, Visual Studio con soporte para escritorio .NET, SQL Server LocalDB y Git. Inno Setup permite compilar el script de distribución. Un navegador permite verificar HTML y realizar impresión de comprobantes.

La solución utiliza un archivo .slnx. Las pruebas son un ejecutable de consola con Windows Forms, no una suite xUnit o NUnit. Antes de ejecutar pruebas de integración, debe comprobarse el catálogo configurado y la protección que impide reutilizar una base temporal existente.

# 4 Estructura de la solución

| Ruta | Responsabilidad |
|---|---|
| PranaElectronica.slnx | Agrupa los cuatro proyectos de aplicación. |
| Prana.Presentacion.csproj y formularios raíz | Presentación, navegación y recursos gráficos. |
| Prana.negocio/ | Servicios CN_, sesión y permisos. |

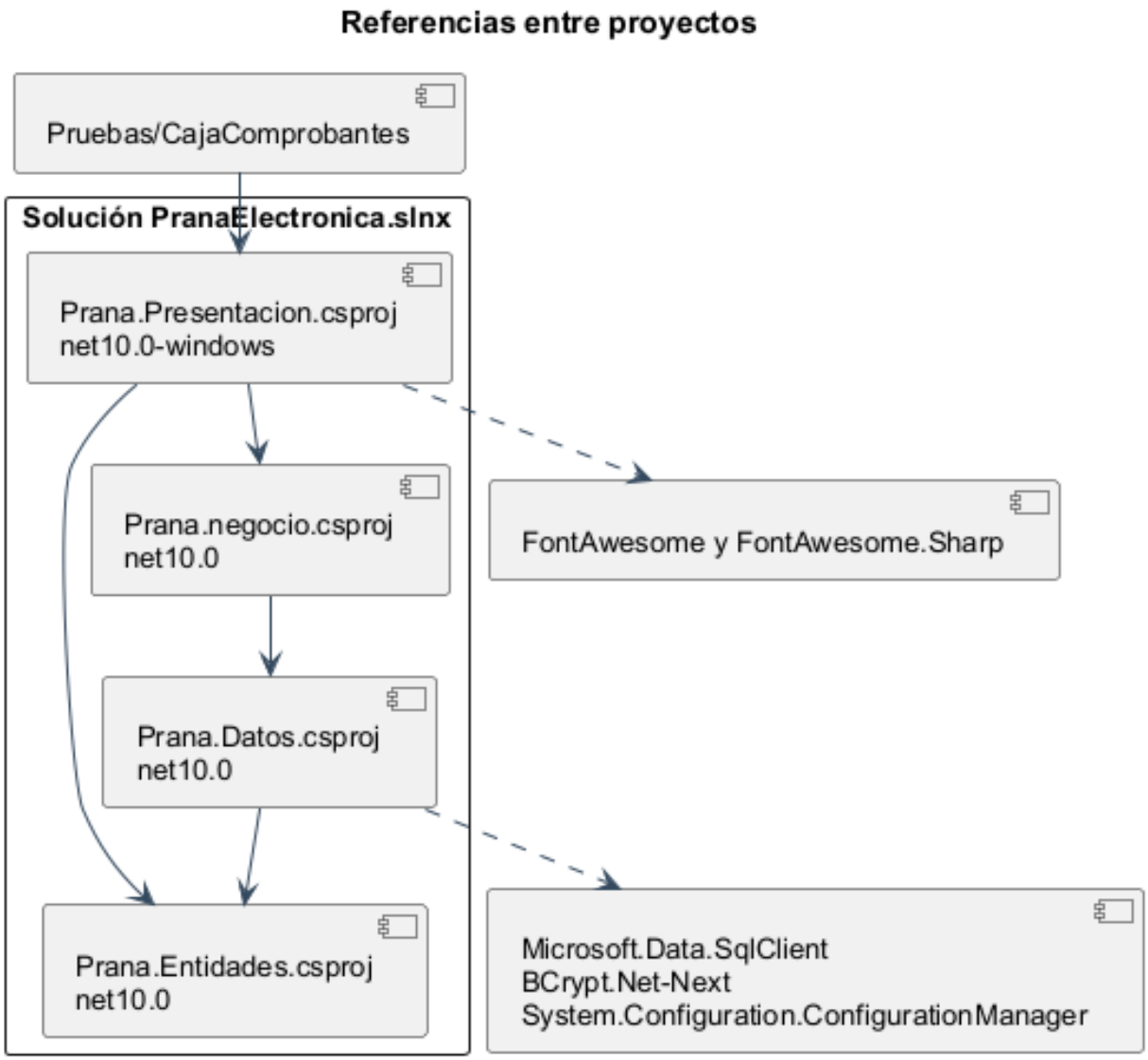
| Ruta | Responsabilidad |
|---|---|
| Prana.Datos/ | Servicios CD_, Conexion, inicializador y utilidades SQL. |
| Prana.Entidades/ | Entidades comerciales y solicitudes. |
| BaseDeDatos/ | Tres scripts activos y documentación previa de inicialización. |
| Properties/PublishProfiles/FolderProfile.pubxml | Perfil de publicación Release autocontenido. |
| Instalador/PranaActual.iss | Script del instalador del repositorio. |
| Pruebas/CajaComprobantes/ | Ejecutable de pruebas de integración y presentación. |
| Plantillas/logo-prana.txt | Recurso de logo utilizado en comprobantes. |

Los directorios bin y obj contienen compilaciones, publicaciones y material de verificación. No sustituyen a los archivos fuente ni deben utilizarse para determinar el alcance funcional por su nombre o fecha solamente.

# 5 Referencias entre proyectos

Prana.Presentacion referencia directamente Prana.Entidades y Prana.negocio. Prana.negocio referencia Prana.Datos. Prana.Datos referencia Prana.Entidades. El proyecto de pruebas referencia Prana.Presentacion y utiliza sus dependencias transitivas.

La configuración de presentación excluye de compilación los archivos fuente de las carpetas Prana.Datos, Prana.Entidades, Prana.negocio y Pruebas. Así evita compilar dos veces archivos de los proyectos incluidos debajo de la raíz.

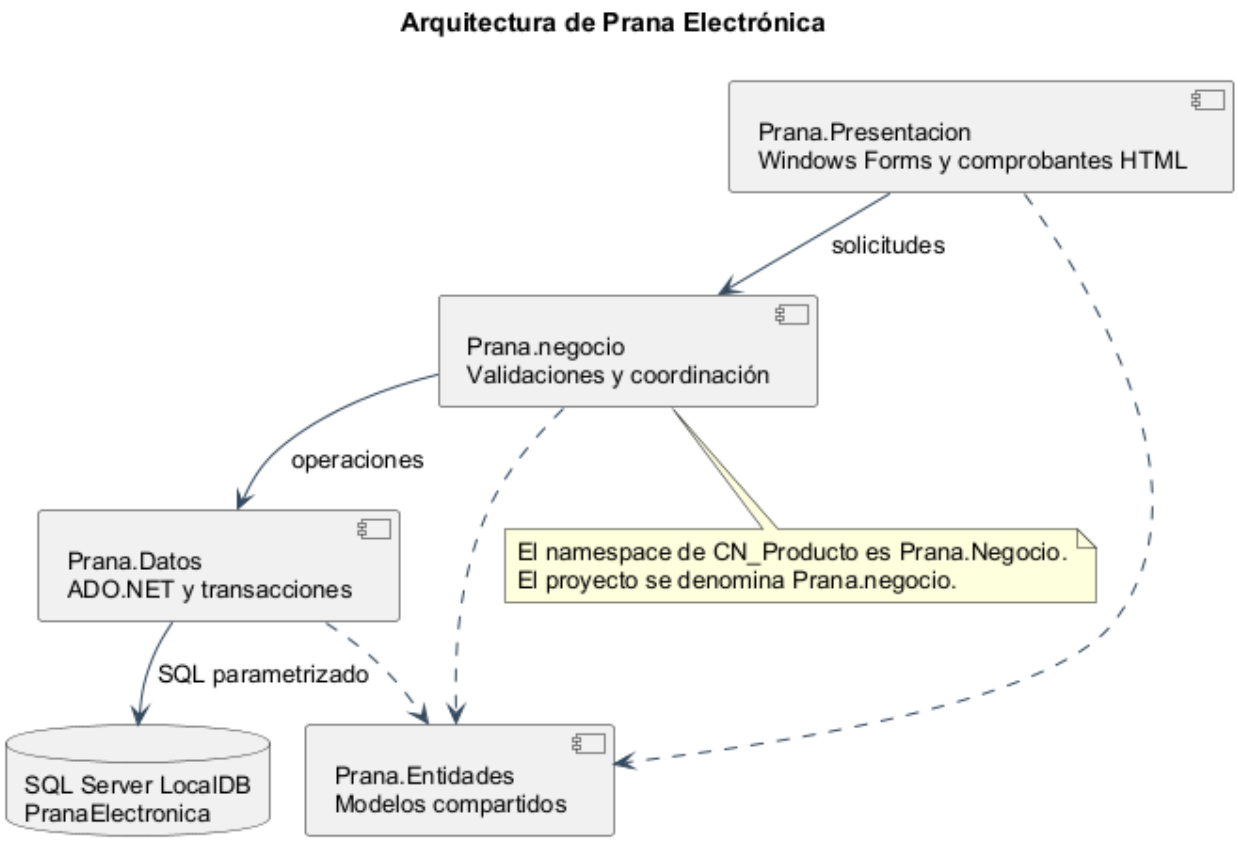



Componentes y paquetes

# 6 Arquitectura por capas

Presentación reúne datos, configura controles y muestra resultados. Negocio aplica reglas de validación y cálculo. Datos abre conexiones, ejecuta SQL y materializa entidades. Entidades representa solicitudes y resultados sin dependencia de los formularios.

CN_RegistroVenta es un ejemplo de coordinación: calcula total base, recargo y desglose de cobro, valida NuevaVenta y llama a CD_RegistroVenta.Guardar. CN_Pago y CN_Entrega verifican permisos antes de las escrituras. La comunicación con SQL Server se realiza exclusivamente desde las clases de datos y el inicializador.




Arquitectura por capas

# 7 Convenciones utilizadas

Los servicios de negocio se identifican con CN_ y los de datos con CD_. Los formularios utilizan frm, con las excepciones login e Interfaz. Las tablas mantienen nombres como Variante_Producto y Detalle_Venta. Las propiedades de C# no tienen una uniformidad total: usuario y producto conservan nombres de clase en minúscula, y los métodos buscarCliente y NuevoCliente utilizan capitalizaciones distintas.

El proyecto de negocio y su carpeta se llaman Prana.negocio, mientras CN_Producto pertenece al namespace Prana.Negocio. Debe respetarse esa diferencia al resolver referencias. CN_VarianteProducto y CD_VarianteProducto son clases vacías; las operaciones activas sobre variantes están implementadas en CN_Producto y CD_Producto.

# 8 Configuración de conexión

App.config define ConexionPrana con Data Source=(localdb)\MSSQLLocalDB, Initial Catalog=PranaElectronica, Integrated Security=True, Encrypt=False y Connect Timeout=30. Conexion.ObtenerCadena lee esa entrada y produce un error si falta o está vacía.

La autenticación integrada utiliza la cuenta de Windows; no se incluye una contraseña SQL. Encrypt=False está configurado para la conexión local. No debe trasladarse esta configuración automáticamente a un servidor remoto sin redefinir seguridad y despliegue.

El inicializador construye su propia conexión fija a LocalDB y PranaElectronica. Cambiar solamente App.config no reconfigura su destino. Debo unificar ambos mecanismos antes de admitir otras instancias o bases.

# 9 Inicializador de base de datos

Program.Main configura Windows Forms y llama a InicializadorBaseDatos.AsegurarBaseDeDatos. Si falla, muestra Error de inicialización y termina antes del login. El inicializador crea la carpeta de datos en %LOCALAPPDATA%/PranaElectronica/Data y, si DB_ID no identifica la base, ejecuta CrearBaseDatos para generar MDF y LDF.

PrepararEsquema recibe una conexión abierta. Inicia una transacción y utiliza sys.sp_getapplock con recurso Prana.ActualizarEsquema y dueño de transacción. Si la base no posee tablas de usuario, ejecuta el script completo. Luego aplica los dos scripts de actualización y confirma; un error revierte la preparación del esquema.

LeerScript busca archivos junto al ejecutable, dentro de BaseDeDatos y en rutas relativas de desarrollo. EjecutarScript divide el contenido por líneas GO y ejecuta cada lote dentro de la misma transacción. La comprobación de esquema vacío no equivale a detectar cualquier base parcialmente incompleta: las migraciones cubren los cambios expresamente programados.

# 10 Scripts SQL

| Script | Uso |
|---|---|
| CrearEstructuraPrana_LocalDB.sql | Crea tablas, restricciones, relaciones y datos iniciales; incluye cambios iniciales de estructura. |
| ActualizarEstructuraPrana_LocalDB.sql | Añade recargos, activo, capacidad de descuento, LogActividad, Venta_Entrega, correo informado único y rol Vendedor. |
| CajaYComprobantes.sql | Añade clasificación de caja, cuenta destino, tablas de movimientos y cierres, copia de comprobantes, código postal y recuperación de actividad de ventas. |

Las migraciones comprueban existencia de tablas y columnas. CajaYComprobantes ajusta el inicio de numeración de ventas a 2500 si corresponde, sin renumerar ventas existentes ni retroceder numeraciones superiores. La ejecución repetida forma parte de las pruebas de integración.

El proyecto de presentación copia BaseDeDatos/*.sql al output mediante CopyToOutputDirectory=PreserveNewest. El script Inno agrega también una carpeta BaseDeDatos. La publicación debe contener los tres archivos y sus versiones vigentes.

# 11 Modelo de datos

El esquema contiene 32 tablas considerando las extensiones de historial, entrega, comprobantes y caja. El diagrama entidad-relación conserva las claves declaradas y diferencia las tablas operativas de las previstas por el modelo.

| Grupo | Tablas principales |
|---|---|
| Clientes y catálogo | Cliente, Localidad, Categoria, Marca, Producto, Variante_Producto. |
| Compras e ingresos | Proveedor, Ingreso_Stock, Detalle_Ingreso. |
| Ventas y cobros | Venta, Detalle_Venta, Pago, Medio_Pago, Estado_Venta, Modalidad_Entrega. |
| Preparación y logística | Remito, Detalle_Remito, Envio, Estado_Envio, Tipo_Envio, Estado_Variante_Producto. |

| Grupo | Tablas principales |
|---|---|
| Copias y caja | Venta_Entrega, Venta_Comprobante, Caja_Movimiento, Caja_Cierre, LogActividad. |
| Acceso y otras estructuras | Usuario, Rol, Formato_Documento, Recibo, Devolucion, Tipo_Resolucion. |

Venta_Comprobante y Venta_Entrega tienen como clave primaria id_venta y referencia a Venta, por lo que representan como máximo un registro de cada tipo por operación. Caja_Cierre contiene una fecha única y datos JSON. Las tablas de detalles implementan relaciones de uno a muchos. La presencia de Recibo y Devolucion no habilita por sí misma un procedimiento comercial en pantalla.

El archivo Diagramas/03_Modelo_entidad_relacion.svg permite consultar el modelo completo a mayor escala. El PDF del diagrama conserva ese tamaño para lectura independiente.

# 12 Entidades principales

| Clase | Uso |
|---|---|
| Cliente y Localidad | Datos de contacto, ubicación y consulta de deuda. |
| producto y VarianteProducto | Catálogo, condición, precio, costo y existencias consultadas. |
| usuario y Rol | Identidad y rol de la cuenta; activo interviene en el login. |
| NuevaVenta, DetalleNuevaVenta y PagoNuevaVenta | Solicitud completa de registro y separación entre base y recargo. |
| venta y ResumenVentas | Resultados del panel y de las consultas de saldo. |
| CobroVenta y PagoRegistrado | Pago posterior cotizado y pago ya persistido. |
| ProductoPorEntregar y EntregaRegistrada | Pendientes de preparación y envíos preparados. |
| ComprobanteVenta | Copia comercial emitida, con cliente, productos y datos de entrega. |
| MovimientoCaja, CierreCaja y ConteoBillete | Movimientos, cierre y cálculos auxiliares; el formulario no ofrece conteo de billetes. |
| FiltroInformeVentas, InformeVenta y AlertaPagoPendiente | Filtros y resultados de consultas. |
| LogActividad | Evento con usuario, acción, entidad, detalle y fecha. |

ProductoPorEntregar.Pendientes calcula Vendidos menos Preparados. EntregaRegistrada.NumeroRemito devuelve IdVenta: el número comercial es distinto del identificador técnico IdRemito. CierreCaja calcula ingresos y gastos por moneda y medio; sus cálculos de contado no implican una función visible de arqueo en la pantalla actual.

# 13 Servicios de negocio

CN_Cliente valida los datos y delega en CD_Cliente; el control de DNI utiliza ExisteClientePorDNI. CN_Producto valida alta e ingreso y coordina operaciones de variantes. CN_RegistroVenta calcula total base y recargos y controla la consistencia de la solicitud.

CN_Pago.Preparar cotiza un CobroVenta y Registrar incorpora el usuario de sesión. CN_Entrega.Preparar y Confirmar aplican GestionarEntregas. CN_Caja exige GestionarCaja en todas sus entradas públicas. CN_Comprobante.Obtener exige acceso al panel antes de recuperar el documento.

CN_MedioPago valida nombre, clasificación y recargos. CN_InformeVentas expone Buscar, ObtenerOpciones y ObtenerAlertas. CN_LogActividad registra eventos y devuelve consultas filtradas. CN_Usuario crea y modifica cuentas y verifica BCrypt durante el acceso. Estos servicios no tienen una aplicación uniforme de permisos; el anexo identifica los límites.

# 14 Acceso a datos

Los servicios CD_ utilizan SqlConnection, SqlCommand, SqlDataReader y SqlTransaction. Las consultas reciben valores como parámetros. GestionVentaSql.Comando construye un comando ligado a conexión y transacción; Catalogo resuelve valores con UPDLOCK y HOLDLOCK y los crea cuando faltan.

ValidarVenta bloquea el encabezado y rechaza ventas inexistentes o con estado iniciado en Anulad o Cancelad. RegistrarActividad conserva eventos de venta dentro de la transacción operativa. CD_Venta.SaldosSql suma monto_final de los pagos antes de relacionar detalles para no multiplicar importes.

Las búsquedas del panel y entregas separan palabras y escapan comodines LIKE. Los informes utilizan filtros parametrizados, pero conservan coincidencias LIKE en los textos suministrados. La actualización de producto desde stock ejecuta dos UPDATE en un comando sin una transacción explícita propia; debe incorporarse una protección equivalente a las altas compuestas.

# 15 Formularios y navegación

Interfaz.AbrirFormulario cierra el formulario activo, configura el nuevo sin borde ni nivel superior y lo inserta en pnlContenido con DockStyle.Fill. El menú utiliza visibilidad por rol para usuarios, medios, informes, historial y alertas. El panel también controla el acceso visible a caja.

Los formularios activos son login, frmPanelPrincipal, frmClientes, frmStock, frmNuevaVenta, frmPagos, frmEntregas, frmCaja, frmMediosPagos, frmInformesVentas, frmAlertas, frmHistorial y frmUsuarios. frmDeudaCliente y frmSeleccionarProducto complementan los procesos.

DistribucionAdaptable, PantallasAdaptables y EstiloGestion organizan controles y grillas. La adaptación a anchos menores emplea desplazamiento y filas que distribuyen controles; las pruebas de presentación comprueban anchos determinados, sin equivaler a una validación de todas las resoluciones.

# 16 Sesión roles y permisos

SesionActual.IniciarSesion guarda un usuario y CerrarSesion establece null. No existe almacenamiento de sesión persistente ni token remoto. Permisos.TienePermiso define Administrador=1 y Vendedor=2; Administrador devuelve acceso completo, mientras Vendedor dispone de una lista fija de capacidades.

CN_Pago.Verificar comprueba primero sesión no nula y luego permiso; CN_Entrega, CN_Caja y CN_Comprobante reutilizan ese control. Permisos.TienePermiso por sí mismo utiliza Usuario con operador de supresión de null y presupone una sesión iniciada. Su uso aislado antes de autenticar puede producir un error.

Los cambios de rol y nombre no actualizan automáticamente el objeto de la sesión actual. Debo incorporar invalidación o renovación controlada y la protección de la última cuenta administradora. La edición de usuarios no contiene controles de activo en pantalla.

# 17 Seguridad con BCrypt

CN_Usuario.GenerarHash rechaza contraseña vacía o compuesta solo por espacios y llama a BCrypt.Net.BCrypt.HashPassword. IniciarSesion busca la cuenta, comprueba activo y ejecuta BCrypt.Net.BCrypt.Verify; no descifra el valor almacenado.

ModificarUsuario utiliza un hash nuevo únicamente cuando recibe una contraseña no vacía. CD_Usuario usa COALESCE para conservar el valor existente cuando recibe null. La lista de usuarios de edición consulta identificador, nombre y rol; no carga la contraseña almacenada.

La validación de unicidad de nombre todavía está pendiente. Usuario posee la protección de correo informado mediante un índice filtrado, pero nombre_usuario no tiene una restricción equivalente. La prueba de edición confirmó que es posible repetir el nombre. Debo resolver esta condición antes de garantizar identificación única por nombre.

# 18 Transacciones

| Operación | Protección implementada |
|---|---|
| Alta de cliente y modificación | Transacción para cliente y localidad relacionada. |
| Alta de producto | Transacción para marca, categoría, producto y variantes. |
| Ingreso de stock | Transacción para proveedor nuevo, encabezado, detalles, stock y costo. |
| Registro de venta | Transacción para catálogos, venta, detalles, pagos, entrega, copia y actividad. |
| Pago posterior | Serializable, validación de venta, saldo, medio activo y bloqueo diario de caja. |
| Preparación y confirmación | Serializable; cantidad pendiente y descuento condicional de disponible. |
| Movimientos y cierre de caja | Transacción y bloqueo común por fecha; rechazo de cierre repetido. |
| Preparación de esquema | Transacción y bloqueo Prana.ActualizarEsquema. |

Al confirmar una venta no se descuenta ni reserva stock en SQL. La comprobación al agregar un producto se realiza sobre datos consultados previamente en la interfaz; requiere una validación transaccional final para cubrir concurrencia.

# 19 Gestión de errores

Los formularios capturan excepciones y presentan mensajes de validación o error. Program.Main finaliza si no puede preparar LocalDB. Los métodos de datos críticos revierten transacciones ante un fallo y vuelven a lanzar la excepción.

CD_Producto.nuevoProducto captura el fallo, revierte y devuelve false, por lo que no conserva el detalle completo del error para el formulario. Otros métodos propagan la causa. Como mantenimiento futuro, debo unificar resultados y excepciones y evitar detalles técnicos innecesarios en mensajes al operador.

La apertura de comprobantes tiene un tratamiento separado: si falla después del registro, la venta sigue guardada. frmNuevaVenta informa el número y permite reimprimir desde el panel sin duplicar el registro.

# 20 Comprobantes HTML

CD_Comprobante.Guardar consulta datos ya persistidos y almacena ComprobanteVenta serializado en Venta_Comprobante dentro de la transacción de venta. Obtener usa esa copia o reconstruye ventas antiguas; CompletarEntrega completa campos que faltaban en emisiones previas sin sustituir importes, cliente y productos originales.

DocumentosPrana.VentaHtml genera el HTML y escapa valores con WebUtility.HtmlEncode. El logo proviene del recurso Prana.LogoComprobante. VarianteImpresa retira la condición Primera al final del texto y conserva otras condiciones. GuardarVenta crea archivos Garantia-número.html y Remito-número.html. MostrarVenta ofrece los botones de apertura e impresión.

GuardarCierre y CierreHtml generan el cierre. Cada venta utiliza Documentos/PranaElectronica/Comprobantes/n?mero. Los cierres se guardan en Documentos/PranaElectronica/Cierres. La impresión PDF corresponde al navegador; la aplicación no produce un PDF fiscal ni una factura electrónica.

# 21 Pruebas

Pruebas/CajaComprobantes/Program.cs crea una base temporal únicamente si no existe y comprueba que ConexionPrana apunta al catálogo de pruebas. El bloque finally libera conexiones y elimina la base que creó. No debe cambiarse esa protección para ejecutar contra la base comercial.

La suite original se detuvo en su expectativa de ciudad obligatoria, distinta de la interfaz y del servicio actuales. PruebasPresentacion.Ejecutar completó 22 verificaciones. Para continuar los casos sin modificar esas fuentes, una copia de pruebas en obj/DocumentacionQA/Harness adaptó ciudad a opcional e incorporó login, edición de usuarios y entrega; completó 125 verificaciones OK y una no satisfactoria sobre duplicidad de nombre.

El documento 04 conserva entorno, procedimientos, resultados y casos planificados. Una comprobación de métodos no equivale a validar manualmente una pantalla ni a probar una instalación en equipo limpio.

# 22 Publicación

El perfil Properties/PublishProfiles/FolderProfile.pubxml define Release, win-x64, SelfContained=true, PublishSingleFile=false y PublishReadyToRun=false. La carpeta concreta del perfil es personal y debe configurarse en el equipo de mantenimiento.

Comando equivalente, desde la raíz del repositorio:

```
dotnet publish Prana.Presentacion.csproj -c Release -r win-x64 --self-contained true -o obj/InstaladorActual/publicacion
```

Compruebe Prana.Presentacion.exe, sus bibliotecas, el archivo .dll.config y los tres scripts SQL. Abra esa publicación y verifique los cambios esperados antes de empaquetar. No utilice una carpeta anterior por el solo hecho de que contenga un ejecutable válido.

# 23 Instalador

Instalador/PranaActual.iss mantiene AppId=PranaElectronica y AppVersion=1.0.0. Define PublishDir relativo a obj/InstaladorActual/publicacion y genera PranaElectronica-Setup-2026-10-02.exe en Instaladores. Configura instalación x64compatible, idioma español, compresión lzma2 y permisos administrativos.

Los accesos apuntan a Prana.Presentacion.exe. La opción final de ejecución utiliza runasoriginaluser para mantener la cuenta de LocalDB. El instalador no incluye un componente de instalación de SQL Server LocalDB.

El script almacenado en el repositorio no incorpora ignoreversion en los archivos principales. Si las bibliotecas conservan la misma versión, debo revisar el reemplazo de archivos durante una actualización. Las variantes del script utilizadas fuera del repositorio no modifican automáticamente este archivo.

# 24 Actualización del sistema

1. Respaldar la base con herramientas de SQL Server y conservar una copia verificada.

2. Compilar y probar el código que se distribuirá.

3. Regenerar la publicación Release y comprobar sus funcionalidades.

4. Compilar el script Inno correspondiente a esa carpeta.

5. Cerrar la aplicación y actualizar en una instalación de prueba antes de la distribución comercial.

6. Comprobar las versiones de archivos, inicio, datos conservados y migraciones.

Mantener AppId permite continuidad de instalación. La versión 1.0.0 fija requiere una política de versionado para identificar entregas y facilitar el reemplazo de bibliotecas. La documentación no afirma que el instalador existente incluya los cambios de usuarios y buscador agregados después de su compilación.

# 25 Respaldo y recuperación

La aplicación no tiene una pantalla de respaldo o restauración. Debe utilizarse un procedimiento externo de SQL Server, operado con la cuenta de Windows que administra LocalDB. Un respaldo debe incluir la base íntegra, verificarse y almacenarse fuera de la computadora de trabajo según la política que se defina.

No debe copiarse el MDF o LDF con la base en uso como sustituto de un respaldo consistente. Antes de una restauración se deben cerrar conexiones y probar el archivo en un catálogo aislado. El destino de los archivos, los permisos de Windows y la configuración del inicializador deben coincidir con el entorno restaurado.

Estos pasos constituyen el procedimiento de mantenimiento previsto; no se presenta una restauración como prueba ejecutada. Debo incorporar una herramienta guiada y un ensayo periódico de recuperación.

# 26 Limitaciones técnicas

Persisten nombres de usuario potencialmente duplicados, permisos no uniformes, ausencia de reserva de stock al confirmar venta y falta de invalidación de sesión después de modificar cuentas. Las actualizaciones de producto no cuentan con la misma cobertura transaccional que las altas.

El inicializador utiliza base y conexión fijas. Los informes calculan saldo con una consulta propia, mientras el panel centraliza reglas de saldo y estados; debo armonizar el tratamiento de estados anulados en futuras operaciones de cancelación. Las alertas toman como vencimiento un mes desde el primer pago y no cubren ventas sin primer cobro.

Las advertencias de compilación incluyen nulabilidad y controles sin uso. Los archivos de prueba con reglas anteriores requieren mantenimiento. Los métodos auxiliares de conteo de billetes existen, pero no deben documentarse como un arqueo disponible en el formulario actual.

# 27 Mantenimiento futuro

Priorizaré unicidad de usuarios, controles de permisos en servicios, validación y reserva transaccional de existencias y consistencia del versionado del instalador. Luego ampliaré cancelaciones, devoluciones, recuperación de contraseña y respaldo integrado.

La cobertura de auditoría debe crecer para altas y modificaciones de clientes, productos, medios y usuarios. Las pruebas deben separar cálculo, persistencia y presentación, mantener datos ficticios y ejecutarse en entornos reproducibles. Los diagramas PlantUML incluidos son fuentes editables y deben actualizarse cuando cambien métodos, relaciones o responsabilidades.